Convolution Exam Guide

Everything You Need Using Your Offline 1 Code

Quick Reference & Concept Guide

Contents

1 Your Existing Code: Quick Reference

Before anything, let us list what you **already have** from Offline 1.

Your Existing Tools (Do NOT rewrite these)

DiscreteSignal class:

- `DiscreteSignal(start, end)` — creates signal with zeros
- `.set_value_at_time(t, val)` — set one sample
- `.get_value_at_time(t)` — get one sample (returns 0 if outside range)
- `.shift(k)` — delay signal by k: $y[n] = x[n-k]$
- `.multiply(c)` — scale all values by c
- `.add(other)` — add two signals sample-by-sample
- `.nonzero_samples()` — list of (t, value) pairs
- `.times()` — range of time indices
- `.plot(title)` — stem plot

LTISystem class:

- `LTISystem(h)` — store impulse response $h[n]$
- `.output(x)` — convolve x with h, return y
- `.output_by_superposition(x)` — same result, different method
- `.output_at_time(x, n)` — compute one sample $y[n]$
- `.output_range(x)` — get (y_{start}, y_{end})

Helper functions:

- `make_signal(start, end, values)` — quick signal creation
- `max_absolute_difference(sig1, sig2)` — compare two signals
- `print_signal(sig, name)` — print signal values

2 Core Concept: What is Convolution?

Convolution in One Sentence

Convolution computes the output of an LTI system for a given input.

2.1 The Formula

$$y[n] = \sum_{k=-\infty}^{\infty} x[k] \cdot h[n-k]$$

In words:

1. Take each input sample $x[k]$
2. Multiply it with $h[n-k]$ (shifted and flipped h)
3. Sum all products → that gives you $y[n]$

2.2 Output Range

Output Range Rule

If $x[n]$ is defined from a to b, and $h[n]$ from c to d:

$$y_{start} = a + c, \qquad y_{end} = b + d$$

2.3 In Your Code

```
system = LTISystem(h)
y = system.output(x)     # this IS convolution
```

That is it. One line does the whole thing.

3 All Operations Using Your Code

Cheat Sheet: Signal Operations

1. Create a signal:

```
x = make_signal(0, 3, [1, 2, 3, 4])     # x[0]=1, x[1]=2, x[2]=3, x[3]=4
```

2. Subtraction (no subtract method, so add + multiply):

```
# a[n] - b[n]
result = a.add(b.multiply(-1))
```

3. First difference $y[n] = x[n] - x[n-1]$:

```
result = x.add(x.shift(1).multiply(-1))
```

Why? `x.shift(1)` gives $x[n-1]$. Multiply by -1 and add.

4. Delay a signal by k**:**

```
x_delayed = x.shift(k)     # y[n] = x[n-k]
```

5. Scale a signal:

```
x_scaled = x.multiply(3)  # every sample * 3
```

6. Add two signals:

```
result = a.add(b)           # sample-wise addition
```

7. Convolve two signals:

```
result = LTISystem(h).output(x)    # y = x * h
```

8. Compare two signals:

```
diff = max_absolute_difference(y1, y2)  # should be ~0
```

4 Question Type 1: Step Response

4.1 The Problem

You are given step response $s[n]$ instead of impulse response $h[n]$. You must find the output $y[n]$ for input $x[n]$.

4.2 Key Relationships

Step Response Formulas

$$h[n] = s[n] - s[n-1] \quad \text{(recover } h \text{ from } s\text{)}$$
$$\Delta x[n] = x[n] - x[n-1] \quad \text{(first difference of } x\text{)}$$
$$y[n] = (\Delta x * s)[n] \quad \text{(output using step response)}$$

4.3 Why It Works

$$\begin{aligned} y &= x * h \\ &= x * (s - s_{delayed}) \\ &= (x - x_{delayed}) * s \quad \text{(distributive law)} \\ &= \Delta x * s \end{aligned}$$

4.4 What to Add to Your Code

Add this ONE function:

```
def first_difference(sig):
    return sig.add(sig.shift(1).multiply(-1))
```

The core calculation (5 lines):

```
h       = first_difference(s)              # h from s
delta_x = first_difference(x)              # delta_x from x
ys      = LTISystem(s).output(delta_x)     # output using s
yh      = LTISystem(h).output(x)           # output using h
diff    = max_absolute_difference(ys, yh)  # should be ~0
```

What Happens Step by Step

1. `first_difference(s)` → computes $s[n] - s[n-1]$ → gives you $h[n]$
2. `first_difference(x)` → computes $x[n] - x[n-1]$ → gives you $\Delta x[n]$
3. `LTISystem(s).output(delta_x)` → convolves Δx with s → gives y_s
4. `LTISystem(h).output(x)` → convolves x with h → gives y_h
5. Both y_s and y_h must be equal

5 Question Type 2: Combination of Systems

5.1 The Problem

Multiple LTI systems are connected in **parallel** and/or **series**. Find the output.

5.2 Two Rules

System Combination Rules

Parallel (outputs are added):

$$h_{eq} = h_1 + h_2$$

In code: `h_eq = h1.add(h2)`

Series (output of one feeds into next):

$$h_{eq} = h_1 * h_2 \quad \text{(convolution)}$$

In code: `h_eq = LTISystem(h1).output(h2)`

5.3 Example: Parallel then Series

Diagram:

```
x --> h1 --\
            + --> h3 --> y
x --> h2 --/
```

This means: x goes through h_1 AND h_2 in parallel, outputs are added, then result goes through h_3.

5.4 What to Add to Your Code

Nothing new. Just use existing functions.

Way 1: Block by block

```
y1    = LTISystem(h1).output(x)      # x through h1
y2    = LTISystem(h2).output(x)      # x through h2
y_sum = y1.add(y2)                   # parallel: add outputs
y     = LTISystem(h3).output(y_sum)  # then through h3
```

Way 2: Combined impulse response

```
h_parallel = h1.add(h2)                          # parallel rule
h_combined = LTISystem(h3).output(h_parallel)    # series rule
y = LTISystem(h_combined).output(x)              # single system
```

Verify:

```
diff = max_absolute_difference(y_way1, y_way2)  # ~0
```

Important

Convolving two impulse responses is done the same way as convolving a signal with an impulse response. In your code, both are just `DiscreteSignal` objects.

```
# convolve h1 with h3
LTISystem(h3).output(h1)    # treats h1 as "input signal"
```

5.5 Other Possible Configurations

Common Configurations

Pure Series: $x \to h_1 \to h_2 \to y$

```
h_combined = LTISystem(h1).output(h2)
y = LTISystem(h_combined).output(x)
```

Pure Parallel: $x \to h_1$ and $x \to h_2$, add outputs

```
h_combined = h1.add(h2)
y = LTISystem(h_combined).output(x)
```

Feedback-like: $x \to h_1 \to h_2 \to h_3$ all in series

```
h12 = LTISystem(h1).output(h2)        # h1 * h2
h123 = LTISystem(h12).output(h3)      # (h1*h2) * h3
y = LTISystem(h123).output(x)
```

6 Question Type 3: Superposition of Signals

6.1 The Problem

Input is a **weighted sum** of multiple signals:

$$x[n] = a \cdot x_1[n] + b \cdot x_2[n] + \dots$$

Find the output using the linearity property.

6.2 The Concept (Simple Analogy)

Think of a machine f that doubles numbers:

$$f(3) = 6$$
$$f(5) = 10$$

What is $f(2 \cdot 3 + 5)$?
Direct: $f(11) = 22$
Superposition: $2 \cdot f(3) + f(5) = 12 + 10 = 22$
Same answer! LTI systems work exactly like this.

6.3 What to Add

Add SuperSignal class (usually given in template):

```
class SuperSignal:
    def __init__(self):
        self.components = []
    def add(self, signal, coefficient=1.0):
        self.components.append((coefficient, signal))
```

Add this ONE method inside LTISystem:

```
def output_super(self, super_signal):
    result = None
    for coeff, sig in super_signal.components:
        y = self.output(sig)            # convolve each
        y_scaled = y.multiply(coeff)    # scale it
        if result is None:
            result = y_scaled
        else:
            result = result.add(y_scaled)  # add to total
    return result
```

Core calculation:

```
# Recipe: x = 2*x1 - x2
X = SuperSignal()
X.add(x1, 2)
X.add(x2, -1)

# Way 1: superposition
y_super = system.output_super(X)

# Way 2: build x directly, then convolve
x_direct = x1.multiply(2).add(x2.multiply(-1))
y_direct = system.output(x_direct)

# verify
diff = max_absolute_difference(y_super, y_direct)
```

What `output_super` Does

For $x = 2x_1 - x_2$:

1. Take x_1, convolve with $h \to y_1$
2. Multiply y_1 by $2 \to 2y_1$
3. Take x_2, convolve with $h \to y_2$
4. Multiply y_2 by $-1 \to -y_2$
5. Add: $2y_1 + (-y_2) = 2y_1 - y_2$

6. Return result

7 Other Possible Question Types

7.1 Type 4: Verify Convolution Properties

Properties of Convolution

Commutative: $x * h = h * x$

```
y1 = LTISystem(h).output(x)
y2 = LTISystem(x).output(h)     # swap roles!
# y1 == y2
```

Associative: $(x * h_1) * h_2 = x * (h_1 * h_2)$

```
y1 = LTISystem(h2).output(LTISystem(h1).output(x))
h_combined = LTISystem(h1).output(h2)
y2 = LTISystem(h_combined).output(x)
# y1 == y2
```

Distributive: $x * (h_1 + h_2) = x * h_1 + x * h_2$

```
y1 = LTISystem(h1.add(h2)).output(x)
y2 = LTISystem(h1).output(x).add(LTISystem(h2).output(x))
# y1 == y2
```

7.2 Type 5: Find Output for Shifted/Scaled Input

Time Invariance and Linearity

If $y[n]$ is the output for $x[n]$, then:

- Output for $x[n-k]$ is $y[n-k]$ (time invariance)
- Output for $a \cdot x[n]$ is $a \cdot y[n]$ (linearity)

```
y = LTISystem(h).output(x)

# shifted input
x_shifted = x.shift(3)
y_shifted = LTISystem(h).output(x_shifted)
# y_shifted should equal y.shift(3)

# scaled input
x_scaled = x.multiply(5)
y_scaled = LTISystem(h).output(x_scaled)
# y_scaled should equal y.multiply(5)
```

7.3 Type 6: Recover Input from Output

Deconvolution Idea

If you know $y[n]$ and $h[n]$, can you find $x[n]$?
This is hard in general, but if h is simple (like identity or delay), it is easy.
Example: if $h[n] = \delta[n]$ (identity), then $y[n] = x[n]$.
Example: if $h[n] = \delta[n-k]$ (pure delay), then $x[n] = y[n+k]$.

```
x_recovered = y.shift(-k)     # undo the delay
```

8 Quick Tricks to Remember

Memory Tricks

Trick 1: Subtraction

```
# a - b = a + (-1)*b
a.add(b.multiply(-1))
```

Trick 2: First difference (most common new operation)

```
# sig[n] - sig[n-1]
sig.add(sig.shift(1).multiply(-1))
```

Trick 3: Convolution = `LTISystem(h).output(x)`

Whether you are convolving:

- signal with impulse response
- two impulse responses
- delta_x with step response

It is always: `LTISystem(one).output(other)`

Trick 4: Parallel = add, Series = convolve

```
h_parallel = h1.add(h2)                    # parallel
h_series   = LTISystem(h1).output(h2)      # series
```

Trick 5: Always verify with `max_absolute_difference`

```
diff = max_absolute_difference(y1, y2)
print(diff)   # should be ~0
```

Trick 6: Output range

```
y_start = x.start_time + h.start_time
y_end   = x.end_time   + h.end_time
```

9 Summary: What to Add for Each Question Type

Question	What to Add	Core Operation
Step Response	`first_difference()`	`sig.add(sig.shift(1).multiply(-1))`
System Combination	Nothing new	`.add()` and `LTISystem().output()`
Superposition	`output_super()`	Loop: output → multiply → add
Convolution Properties	Nothing new	Swap/regroup `LTISystem().output()`
Shifted/Scaled Input	Nothing new	`.shift(k)` and `.multiply(c)`

10 Final Checklist Before Exam

Before Exam Checklist

1. Make sure you understand what `LTISystem(h).output(x)` does
   - It convolves x with h. That is ALL it does.
2. Remember: subtraction = `.add(.multiply(-1))`
3. Remember: first difference = `sig.add(sig.shift(1).multiply(-1))`
4. Parallel = `.add()`, Series = `LTISystem().output()`
5. Always verify two methods match using `max_absolute_difference()`
6. Output range: $y_{start} = x_{start} + h_{start}$, $y_{end} = x_{end} + h_{end}$
7. Your code handles everything. You just need to **call the right functions in the right order**.